

La relazione presenta spunti interessanti, tipo l'uso del metodo di Welch, ma anche alcune affermazioni inesatte o quantomeno piuttosto deboli. L'algoritmo per l'identificazione della fondamentale è spiegato in modo non sempre chiarissimo, tralasciando alcuni dettagli; è chiaramente una ricetta empirica ma non si forniscono molte giustificazioni. La classificazione dei registri chiaramente è complessa, ma il "trucco" della media mi sembra fallimentare senza un modo per mantenere in qualche modo le classificazioni "uniformi" tra esecuzioni differenti. Esperienza 3 - Gruppo E
Manca una discussione ragionata dei risultati ottenuti. Molte figure sono completamente inutili.
Voto: 7.5/10

Classificazione di note e timbri prodotte da un organo



Alice Vergani

Dave Parmegiani

Giovanni Edoardo Folpini

1 Introduzione

L'obiettivo di questo lavoro è quello di analizzare delle registrazioni audio di note prodotte da un organo per stabilire che note siano e con quale timbro sono state suonate. In particolare, vogliamo scrivere un programma in grado di classificare frequenza e timbro automaticamente, senza il bisogno di ascoltare la registrazione.



Questo programma dev'essere in grado di leggere il file, calcolare lo spettro di potenza, e dai suoi picchi ricavare la frequenza fondamentale e le armoniche successive. La frequenza fondamentale determina di che nota si tratta, mentre i valori di potenza delle armoniche ci danno informazioni sul timbro.

Per distinguere i timbri, usiamo l'algoritmo K-means per il *clustering*. L'idea è che note con lo stesso timbro avranno profili spettrali con proprietà simili, e quindi vogliamo individuare i timbri come *cluster* di note.

2 Descrizione dei dati



Le registrazioni che andiamo ad analizzare ci sono state fornite dal Prof. Lombardi [1]. Si tratta di 280 file in formato `.wav`, contenenti registrazioni di note prodotte da un organo. La lettura di questi file ha rivelato che fossero tutti quanti in formato stereo, con profondità di 4 bit e con frequenza di campionamento di 48 kHz.



Sappiamo due cose a priori di queste registrazioni: la prima è che le note sono prodotte con un totale di 5 timbri diversi; la seconda è che per ciascun timbro non ci sono ripetizioni di una stessa nota. Una conseguenza di queste due informazioni è che ci aspettiamo che nei dati una determinata nota sia presente al più 5 volte.

3 Metodi computazionali

3.1 Calcolo dello spettro di potenza

Per eseguire un'analisi quantitativa delle registrazioni è necessario ottenere delle caratteristiche che le descrivano numericamente. Quando si lavora con segnali audio è tipico ottenere queste caratteristiche dallo spettro di potenza, siccome la trasformata di Fourier è unica per un dato segnale. Nel nostro caso le caratteristiche di interesse sono le frequenze delle armoniche e le potenze



ad esse associate, che corrispondono tipicamente a dei picchi nello spettro di potenza.

Per riuscire a distinguere questi picchi automaticamente è necessario lavorare con uno spettro in cui il rumore di fondo è sufficientemente basso. A tal fine, scegliamo di calcolare lo spettro di potenza usando il metodo di Welch [2]. Il metodo funziona nel modo seguente: divide il segnale in segmenti uguali e parzialmente sovrapposti; applica una funzione finestra a tutti i segmenti; calcola lo spettro di potenza di ciascun segmento; ricava lo spettro finale come la media degli spettri precedentemente calcolati. L'idea è che le armoniche in ciascun segmento rimangono le stesse, quindi facendo la media degli spettri le posizioni dei picchi rimangono invariate, mentre il rumore tende a diminuire. In Fig. 1 mostriamo un confronto tra uno spettro di potenza ottenuto senza e con il metodo di Welch. Nel caso in cui applichiamo il metodo, Fig. 1B, il rumore è quasi del tutto assente fino alla quinta armonica.



Nel codice, impieghiamo il metodo con l'apposita funzione `welch` della libreria Python `scipy` [3]. Scegliamo di dividere il segnale sempre in 10 segmenti, con sovrapposizione di mezza lunghezza tra due segmenti successivi. In ciascun segmento applichiamo una funzione finestra gaussiana, centrata nel segmento e di deviazione $\sigma = 0.2s$. Fissare numero di segmenti e deviazione della finestra implica una risoluzione in frequenza diversa per ogni registrazione, siccome in generale hanno durate diverse. Questo aspetto non rappresenta però un problema per la nostra analisi. Il motivo è che le durate dei segnali sono tali per cui le risoluzioni risultanti sono comprese tra 0.8 Hz e 2.5 Hz. Queste risoluzioni sono sufficienti al corretto riconoscimento delle armoniche.

3.2 Selezione delle armoniche

Calcolato lo spettro di potenza, ne individuiamo i picchi principali usando la funzione `find_peaks` di `scipy`. Per farlo, impostiamo che i picchi debbano avere una prominenza di 25 dB. Una prominenza fissa funziona bene perché anche la risoluzione degli spettri è praticamente fissa. Idealmente, ogni picco rilevato dovrebbe essere un multiplo intero della frequenza fondamentale f_1 , ma in pratica capita che non vengano lette armoniche o venga invece letto rumore. Stimiamo allora f_1 con un algoritmo basato sul Massimo Comun Divisore (MCD).



Un metodo MCD è principalmente reso necessario dal fenomeno della fondamentale mancante. Si tratta di una mancata lettura di f_1 tra i picchi a causa delle caratteristi-

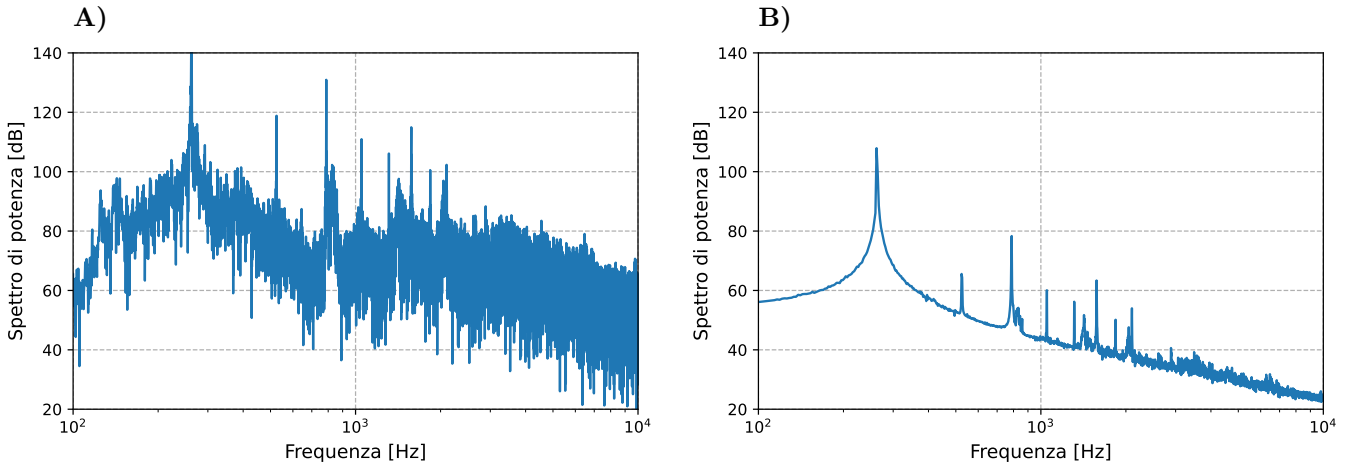


Figura 1: Confronto tra due modi diversi di calcolare lo stesso spettro di potenza. Gli spettri qui presentati sono stati ottenuti per il file `note_111.wav`, scelto arbitrariamente. Le ascisse sono in scala logaritmica, ed è raffigurata solo la sezione di interesse dell'intero spettro: la frequenza fondamentale è 262 Hz, corrispondente a un DO4. **(A)** Spettro ottenuto applicando una singola finestra gaussiana, larga quanto l'intero sostegno della nota. Più precisamente, la deviazione è impostata a $1/6$ della durata del segnale. **(B)** Spettro ottenuto con il metodo di Welch per la riduzione del rumore. I parametri usati sono quelli descritti in Sez. 3.1.

che dell'organo oppure di problemi col segnale. In questo caso, f_1 può ancora essere dedotta correttamente dalla spaziatura costante tra le armoniche successive. Tuttavia, questo calcolo introduce un forte bias sistematico a favore delle basse frequenze: sottomultipli della fondamentale come $f_1/2$ sono anch'essi in grado di giustificare le armoniche rilevate, tendenzialmente ancora meglio di f_1 . Per ovviare a questo problema implementiamo nell'algoritmo una funzione costo. Questa somma i residui con le armoniche trovate ad una penalità proporzionale alle armoniche mancanti. Se per costruzione ciascun residuo è un valore nell'intervallo $[0, 0.5]$, scegliamo indicativamente che la penalità per ciascuna armonica mancante valga 0.3, un valore che sia comparabile a quello di un tipico residuo.

L'algoritmo associa dunque un costo a tutti i candidati, e il candidato col costo minore è selezionato come frequenza fondamentale. Il candidato più grande è il picco più energetico trovato, che non coincide sempre con il primo picco. Il candidato più piccolo invece è $1/3$ del primo picco: questo permette di vagliare la possibilità che sia stata saltata la lettura di fino a due armoniche. Tutti i candidati intermedi sono equispaziati con passo 0.1 Hz. L'algoritmo scansiona i candidati in ordine decrescente, e accetta una nuova frequenza fondamentale solo se riduce il costo del precedente miglior candidato di almeno il 10%. Una volta ottenuta la frequenza fondamentale di un segnale, ricaviamo con immediatezza le armoniche successive.

3.3 Suddivisione in timbri

Le caratteristiche che decidiamo di usare per il clustering sono i valori dello spettro di potenza valutato alle frequenze delle prime sei armoniche. Raccogliendo questi valori in un vettore, i segnali vengono mappati in punti che vivono in $\mathbb{R}^6$. Per individuare e distinguere i cluster formati da questi punti usiamo l'algoritmo K-means. Questo è un semplice metodo di *machine learning* non su-

pervisionato, ed in particolare è un esempio di algoritmo di aspettazione-massimizzazione.

Il funzionamento si basa sull'alternanza di due step fino a convergenza. Per avviare l'algoritmo specifichiamo il numero di cluster K , quindi selezioniamo casualmente K punti da usare come centroidi di partenza. Il primo step, in gergo step E, consiste nel partizionare l'iperspazio delle caratteristiche con una tassellatura di Voronoi, generata dai centroidi attuali. Tassellato lo spazio, l'assegnazione di ogni punto ad un politopo, e quindi cluster, diventa triviale. Il secondo step, in gergo step M, consiste nel calcolare i baricentri di tutti i punti appartenenti ad uno stesso cluster. Una volta calcolati i baricentri, questi vengono scelti come nuovi centroidi. L'algoritmo continua ad alternare step E e step M fino alla convergenza delle posizioni dei centroidi.

Il metodo K-means necessita di operare in un iperspazio con una norma ben definita. Inoltre, per convergere ai risultati corretti, l'algoritmo presume che i cluster siano approssimativamente sferici e di dimensioni simili. Sebbene la seconda condizione sia soddisfatta normalizzando le caratteristiche, la prima condizione resta in generale la limitazione principale dell'algoritmo. Un'altra limitazione importante è che il K-means restituisce un minimo locale invece di quello globale, rendendo necessario eseguire l'algoritmo molte volte. I pregi di K-means, invece, sono un'elevata velocità e una convergenza assicurata (quantomeno nella versione *hard*, che è quella che usiamo noi).

Nel codice, impieghiamo il metodo K-means con l'apposita classe `Kmeans` della libreria Python `sklearn` [4]. Sappiamo a priori che il numero K di cluster nei dati è 5. Scegliamo che l'inizializzazione dei centroidi sia una scelta casuale tra i dati, e che internamente sia usato l'algoritmo di Lloyd. Se specifichiamo un numero di iterazioni, la funzione eseguirà l'algoritmo quel numero di volte, e restituirà il singolo clustering migliore trovato tra

quelle iterazioni. La bontà di un clustering è quantificata dalla varianza intra-cluster. Tramite dei test preliminari, abbiamo deciso che un numero di iterazioni dell'ordine delle migliaia fosse il giusto compromesso tra qualità dei risultati e tempo di computazione.

4 Analisi e discussione

Ogni file viene letto come vettore di valori grazie alle funzioni della libreria Python `wavio` [5]. La lettura di questi vettori rivela che tutte le registrazioni includono una fase di attacco, ma non una di rilascio. Per questo motivo, appena leggiamo un file ne tagliamo il primo 15% e l'ultimo 5%, così da lavorare sul solo sostegno della nota. La scelta di queste percentuali è arbitraria, dato che percentuali simili sembravano dare risultati preliminari analoghi. La successiva analisi segue in generale l'ordine già delineato nella Sezione precedente.

Il primo passo consiste nel calcolo dello spettro di potenza. Per svolgerlo, applichiamo il metodo di Welch, trasformando poi i valori della potenza in decibel. Il vantaggio di impiegare il metodo di Welch è già stato illustrato in Fig. 1.

Il secondo passaggio è la determinazione dei picchi dagli spettri. Selezioniamo solo i picchi con una prominenza fissa di 25 dB, e li analizziamo con il metodo MCD presentato in Sez. 3.2. Una volta ottenute tutte le frequenze fondamentali, possiamo associare ogni registrazione alla nota musicale corrispondente. Scriviamo la lista di frequenze nominali delle note di un pianoforte e le trasformiamo in scala logaritmica in base 2. Facciamo lo stesso anche con le f_1 stimate e le arrotondiamo alla nota nominale più vicina. Il risultato di questa classificazione è mostrato in Fig. 2. Contiamo 56 note diverse e notiamo che tutte le colonne contengono 5 elementi. Le informa-

zioni a priori sulle registrazioni non imponevano che tutte le colonne avessero necessariamente la stessa altezza, però imponevano che tutte le colonne avessero un'altezza al più di 5 elementi. Il risultato in Fig. 2 soddisfa questo prerequisito, quindi conferma la validità dell'analisi svolta finora.

Per classificare ciascuna registrazione in base al timbro, facciamo clustering sulle potenze delle prime sei armoniche. La f_1 stimata è la prima armonica, quindi calcoliamo le armoniche "teoriche" come i multipli interi di f_1 . Se un'armonica teorica è entro una certa tolleranza da un'armonica rilevata, salviamo come caratteristica il picco rilevato; la tolleranza usata è 0.03 in scala logaritmica in base 10 delle frequenze. Altrimenti, valutiamo lo spettro di potenza alla frequenza teorica e salviamo quello come valore. Ogni segnale viene dunque descritto da un vettore 6-dimensionale di potenze. Vogliamo usare il metodo K-means per il clustering, ma questo raggruppa i dati basandosi sulla distanza euclidea: vettori con potenze molto diverse verrebbero considerati distanti anche se appartenenti allo stesso timbro. Per risolvere questo problema, normalizziamo i vettori rispetto alla potenza della frequenza fondamentale di ciascuno, e usiamo questi vettori rinormalizzati come vettori di caratteristiche per il clustering. Eseguendo ora l'algoritmo K-means, il risultato tipico è mostrato in Fig. 3. Evidenziamo in particolare i profili corrispondenti ai centroidi individuati dal clustering: questi profili sono interpretabili come descrizioni caratteristiche di ciascun timbro.

Considerando lo stesso risultato di Fig. 3, la distribuzione delle registrazioni per cluster è mostrata invece in Fig. 4A. Sulla base delle informazioni a priori e di Fig. 2, ci saremmo aspettati che ogni timbro includesse esattamente 56 registrazioni ciascuno. Notiamo invece che la ripartizione restituita dall'algoritmo non è perfettamente bilanciata. Questo è sintomatico di due cose: limitazioni

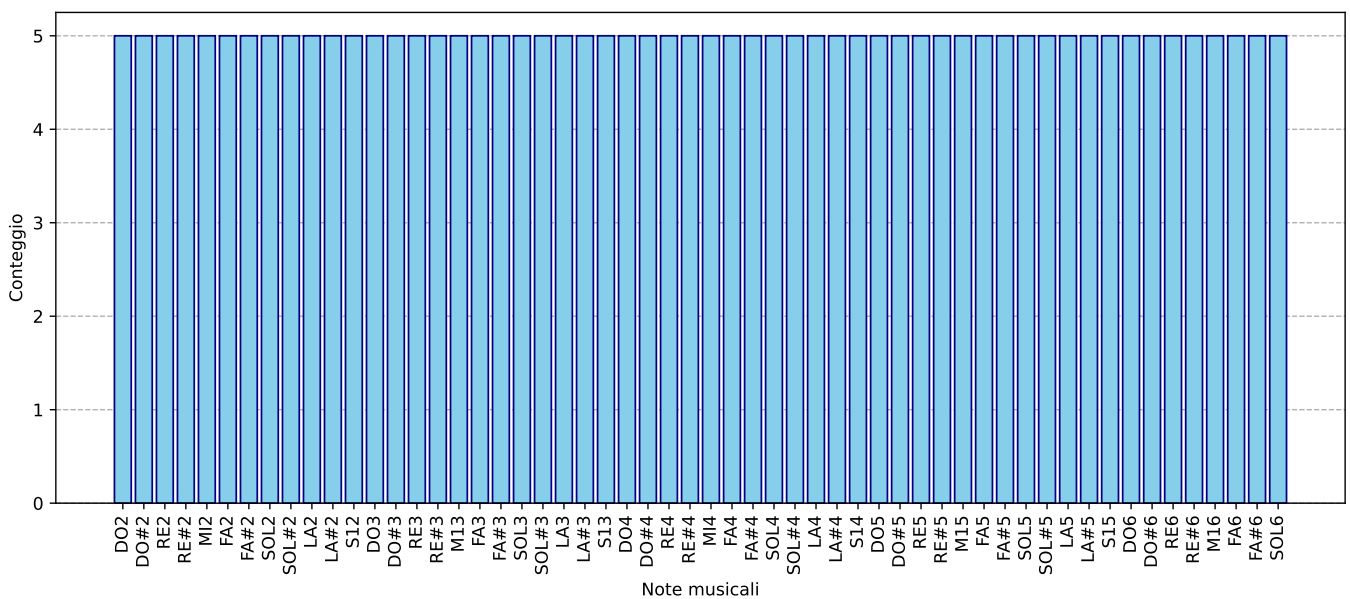


Figura 2: Distribuzione dei segnali audio in base alle loro frequenze fondamentali. Ogni segnale è associato alla nota la cui frequenza nominale è più vicina alla frequenza fondamentale trovata. Si contano in totale 56 note diverse, tutte con 5 registrazioni associate.

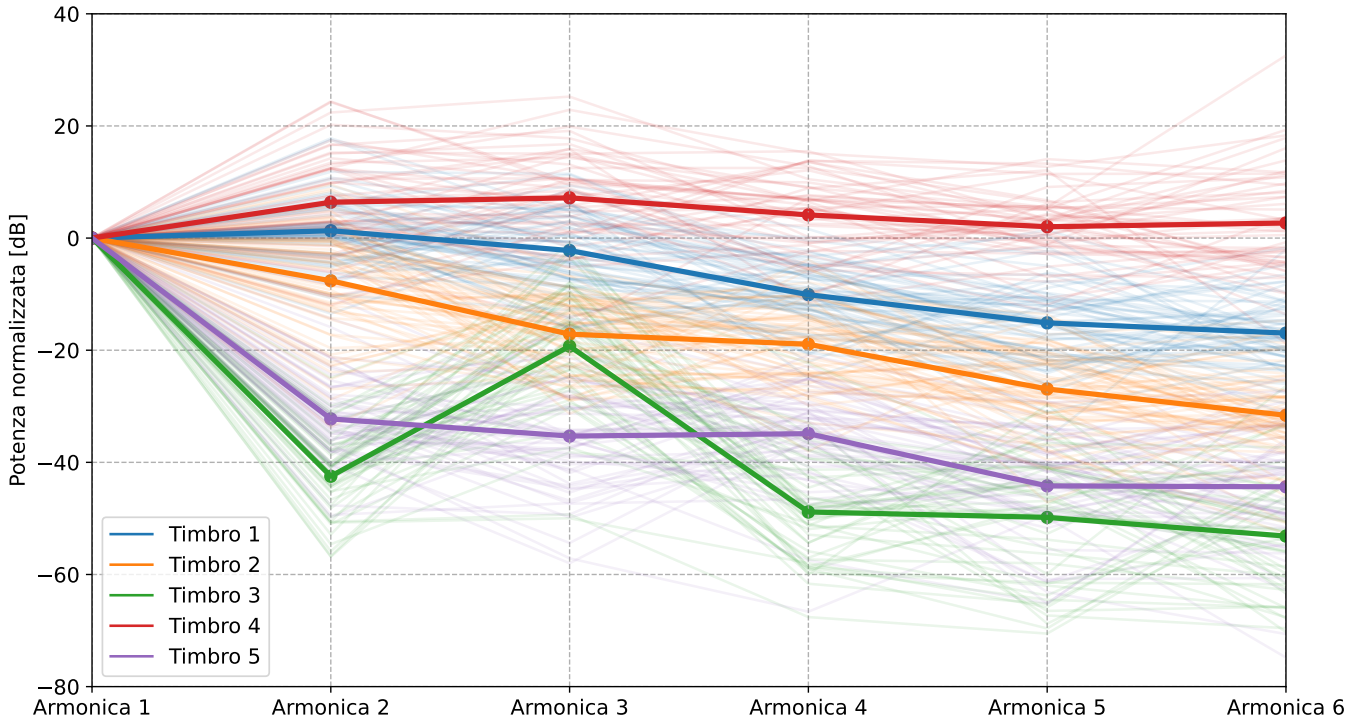


Figura 3: Profili spettrali di tutti i segnali e dei centroidi dei cluster. Le potenze delle armoniche sono normalizzate con la potenza della prima armonica, la frequenza fondamentale. Per questo motivo, tutti i profili partono dallo stesso punto. I profili più chiari sono i profili di tutti i segnali, e sono colorati per appartenenza ai cluster. Le appartenenze ai cluster sono state assegnate da una esecuzione dell'algoritmo K-means. I profili più marcati, invece, corrispondono ai centroidi individuati dall'algoritmo. Questi profili marcati sono interpretabili come caratteristici di ciascun timbro.

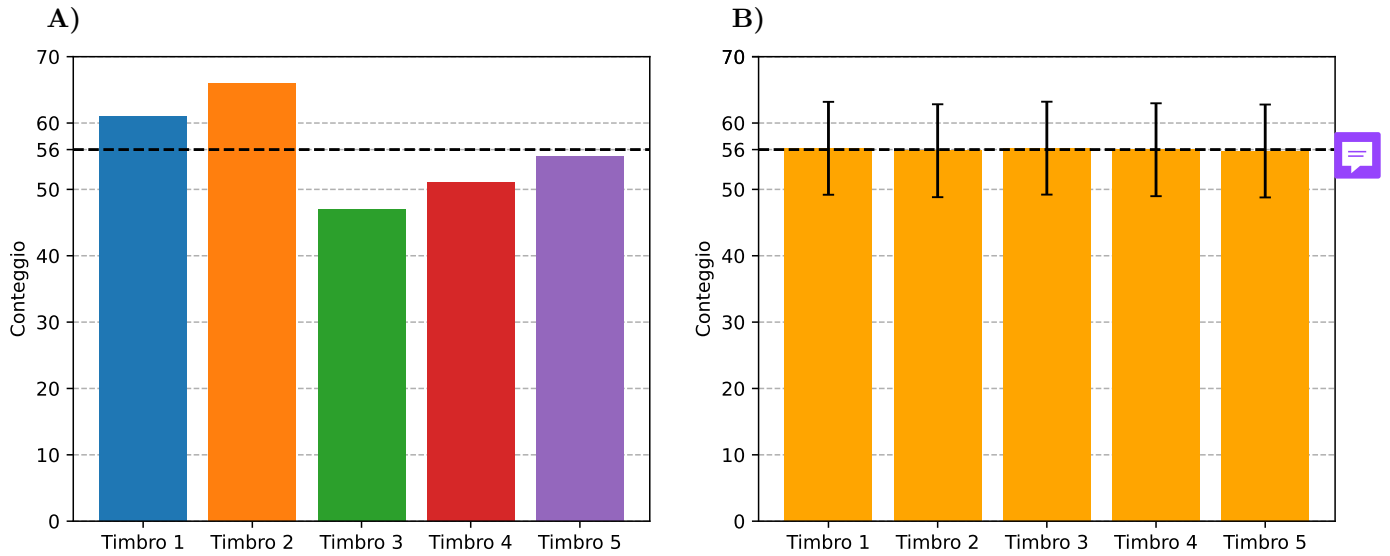


Figura 4: Confronto tra il risultato di una sola esecuzione dell'algoritmo K-means e la media dei risultati di 1000 esecuzioni dello stesso algoritmo. **(A)** Distribuzione di appartenenza delle note ai 5 timbri. Questo è il risultato di una sola esecuzione dell'algoritmo K-means. In particolare, questa distribuzione è quella associata ai risultati già rappresentati in Fig. 3. **(B)** Media delle distribuzioni di appartenenza delle note ai 5 timbri. L'algoritmo K-means è stato eseguito 1000 volte e queste sono le medie ottenute per i numeri di note per timbro. I valori si discostano da 56 solo di qualche punto decimale. Le barre di errore corrispondono ad un intervallo di una deviazione: per tutte le colonne, l'incertezza è ± 7 .

dell'algoritmo e sovrapposizione dei cluster. Le limitazioni del K-means sono state già discusse in Sez. 3.3. La soluzione trovata è solo un minimo locale, che in generale può anche essere molto distante dal clustering ottimale. Infatti, se eseguiamo l'algoritmo 1000 volte e calcoliamo

la media dei numeri di note per cluster, otteniamo il risultato in Fig. 4B. Questa mostra che il clustering ottimale è effettivamente quello che ci aspettavamo, ma ciascuna esecuzione del K-means ci restituisce solo una soluzione approssimata. Le barre di errore in Fig. 4B

corrispondono ad un errore relativo superiore al 10%, rafforzando ulteriormente la volatilità dei risultati delle singole esecuzioni del K-means.

Il motivo per cui questa limitazione del K-means diventa rilevante è che i dati non sono ideali. Il risultato approssimativo del K-means suggerisce infatti che i cluster siano parzialmente sovrapposti. Qualitativamente possiamo vederlo già da Fig. 3, ma per visualizzare meglio i cluster svolgiamo un'Analisi delle Componenti Principali (PCA) [6]. Questo è un metodo di riduzione dimensionale che mappa i dati su nuove coordinate, dette principali, che massimizzano la varianza dei dati. Noi intendiamo usare questo algoritmo solo per una visualizzazione alternativa dei dati. Nel codice, impieghiamo il metodo PCA con l'apposita classe `PCA` di `sklearn`, e trasformiamo i vettori di potenze normalizzate su due componenti principali. Il risultato ottenuto è mostrato in Fig. 5. Sebbene i cluster siano ben distinguibili, ci sono delle evidenti zone di confine in cui tendono a mescolarsi, nonché sovrapporsi. In queste regioni le caratteristiche armoniche di registrazioni appartenenti a timbri diversi risultano essere estremamente simili tra loro. Il K-means si basa unicamente sulla minimizzazione della distanza euclidea, quindi è inevitabile che i punti giacenti su questi confini sfumati vengano assegnati talvolta ad un cluster e talvolta a quello di fianco, di esecuzione in esecuzione.

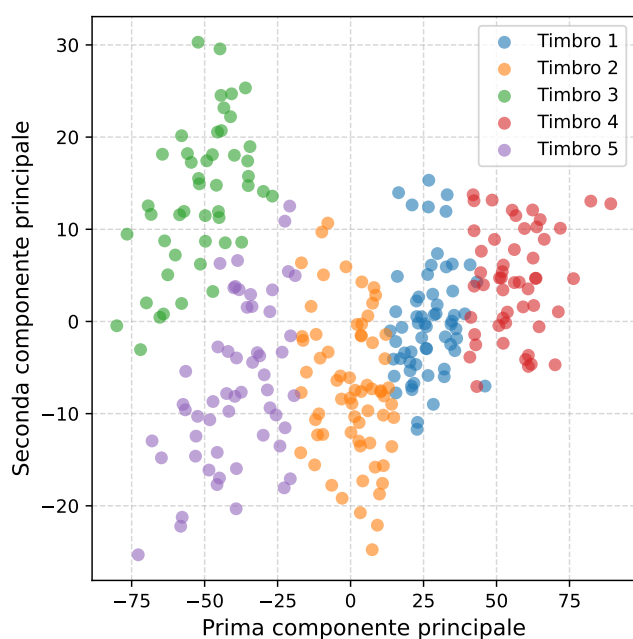


Figura 5: Risultato di una PCA a due componenti dei dati delle potenze normalizzate delle armoniche. Gli assi corrispondono alle prime due componenti principali della PCA: sono le due quantità ausiliarie che evidenziano maggiormente la varianza dei dati. Ogni punto corrisponde a un segnale audio, e sono colorati per appartenenza ai timbri. In particolare, queste appartenenze sono quelle dei risultati già rappresentati in Fig. 3 e Fig. 4A.

5 Conclusioni

La nostra analisi mostra che l'approccio seguito porta ad una corretta identificazione delle note e dei timbri prodotti da un organo. L'impiego del metodo di Welch combinato con un metodo MCD permette di superare problemi quali rumore di fondo e fondamentali mancanti. Ritroviamo correttamente che nelle registrazioni date ci sono 56 note distinte, ognuna con 5 campioni.

L'identificazione dei timbri è ottenuta leggendo e normalizzando le potenze delle prime sei armoniche, su cui poi applichiamo l'algoritmo K-means. La variabilità dei risultati delle singole esecuzioni è dovuta alla sovrapposizione dei cluster, come meglio evidenziato da una PCA. Nonostante questo, la media di più esecuzioni fornisce il risultato corretto, sebbene con una varianza notevole.

L'effettiva assegnazione di una registrazione ad un timbro piuttosto che un altro può venir svolta o affidandosi ad una singola esecuzione del K-means, o mediando l'inclusione nei cluster su numerose esecuzioni. Questa seconda opzione non è inclusa nella nostra analisi, che può dunque venire ampliata.

Riferimenti bibliografici

- [1] M. Lombardi. *Organ*. <https://tinyurl.com/yc7byrbk>. Consultato ad Aprile 2026.
- [2] P. Welch. "The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms". In: *IEEE Transactions on Audio and Electroacoustics* 15.2 (1967), pp. 70–73. DOI: 10.1109/TAU.1967.1161901.
- [3] P. Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature Methods* 17.3 (2020), pp. 261–272. DOI: 10.1038/S41592-019-0686-2.
- [4] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830. URL: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- [5] W. Weckesser. *wavio*. <https://github.com/WarrenWeckesser/wavio>. Consultato ad Aprile 2026.
- [6] M. E. Tipping e C. M. Bishop. "Mixtures of Probabilistic Principal Component Analyzers". In: *Neural Computation* 11.2 (1999), pp. 443–482. DOI: 10.1162/089976699300016728.